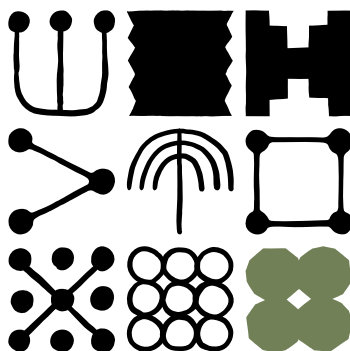


Beyond permission prompts: making Claude Code more secure and autonomous

Anthropic · 2025-10-20 · 原文 / Source



Claude Code's new sandboxing features, a bash tool and Claude Code on the web, reduce permission prompts and increase user safety by enabling two boundaries: filesystem and network isolation.

In Claude Code, Claude writes, tests, and debugs code alongside you, navigating your codebase, editing multiple files, and running commands to verify its work. Giving Claude this much access to your codebase and files can introduce risks, especially in the case of prompt injection.

To help address this, we've introduced two new features in Claude Code built on top of sandboxing, both of which are designed to provide a more secure place for developers to work, while also allowing Claude to run more autonomously and with fewer permission prompts. In our internal usage, we've found that sandboxing safely reduces permission prompts by 84%. By defining set boundaries within which Claude can work freely, they increase security and agency.

Keeping users secure on Claude Code

Claude Code runs on a permission-based model: by default, it's read-only, which means it asks for permission before making modifications or running any commands. There are some exceptions to this: we auto-allow safe commands like `echo` or `cat`, but most operations still need explicit approval.

Constantly clicking "approve" slows down development cycles and can lead to 'approval fatigue', where users might not pay close attention to what they're approving, and in turn making development less safe.

To address this, we launched sandboxing for Claude Code.

Sandboxing: a safer and more autonomous approach

Sandboxing creates pre-defined boundaries within which Claude can work more freely, instead of asking for permission for each action. With sandboxing enabled, you get drastically fewer permission prompts and increased safety.

Our approach to sandboxing is built on top of operating system-level features to enable two boundaries:

1. **Filesystem isolation**, which ensures that Claude can only access or modify specific directories. This is particularly important in preventing a prompt-injected Claude from modifying sensitive system files.
2. **Network isolation**, which ensures that Claude can only connect to approved servers. This prevents a prompt-injected Claude from leaking sensitive information or downloading malware.

It is worth noting that effective sandboxing requires *both* filesystem and network isolation. Without network isolation, a compromised agent could exfiltrate sensitive files like SSH keys; without filesystem isolation, a compromised agent could easily escape the sandbox and gain network access. It's by using both techniques that we can provide a safer and faster agentic experience for Claude Code users.

Two new sandboxing features in Claude Code

Sandboxed bash tool: safe bash execution without permission prompts

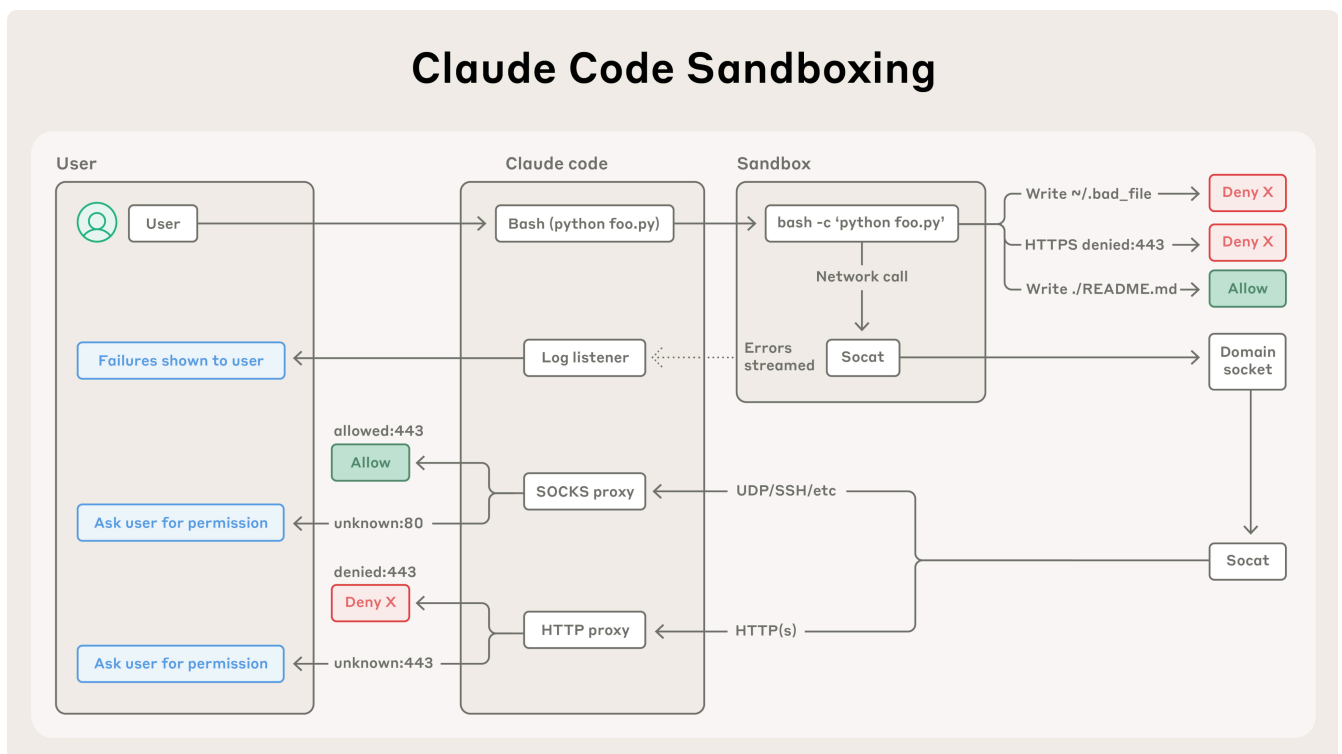
We're introducing a new sandbox runtime, available in beta as a research preview, that lets you define exactly which directories and network hosts your agent can access, without the overhead of spinning up and managing a container. This can be used to sandbox arbitrary processes, agents and MCP servers. It is also available as an open source research preview.

In Claude Code, we use this runtime to sandbox the bash tool, which allows Claude to run commands within the defined limits you set. Inside the safe sandbox, Claude can run more autonomously and safely execute commands without permission prompts. If Claude tries to access something *outside* of the sandbox, you'll be notified immediately, and can choose whether or not to allow it.

We've built this on top of OS level primitives such as Linux bubblewrap and MacOS seatbelt to enforce these restrictions at the OS level. They cover not just Claude Code's direct interactions, but also any scripts, programs, or subprocesses that are spawned by the command. As described above, this sandbox enforces both:

1. **Filesystem isolation**, by allowing read and write access to the current working directory, but blocking the modification of any files outside of it.
2. **Network isolation**, by only allowing internet access through a unix domain socket connected to a proxy server running outside the sandbox. This proxy server enforces restrictions on the domains that a process can connect to, and handles user confirmation for newly requested domains. And if you'd like further-increased security, we also support customizing this proxy to enforce arbitrary rules on outgoing traffic.

Both components are configurable: you can easily choose to allow or disallow specific file paths or domains.



Claude Code's sandboxing architecture isolates code execution with filesystem and network controls, automatically allowing safe operations, blocking malicious ones, and asking permission only when needed.

Sandboxing ensures that even a successful prompt injection is fully isolated, and cannot impact overall user security. This way, a compromised Claude Code can't steal your SSH keys, or phone home to an attacker's server.

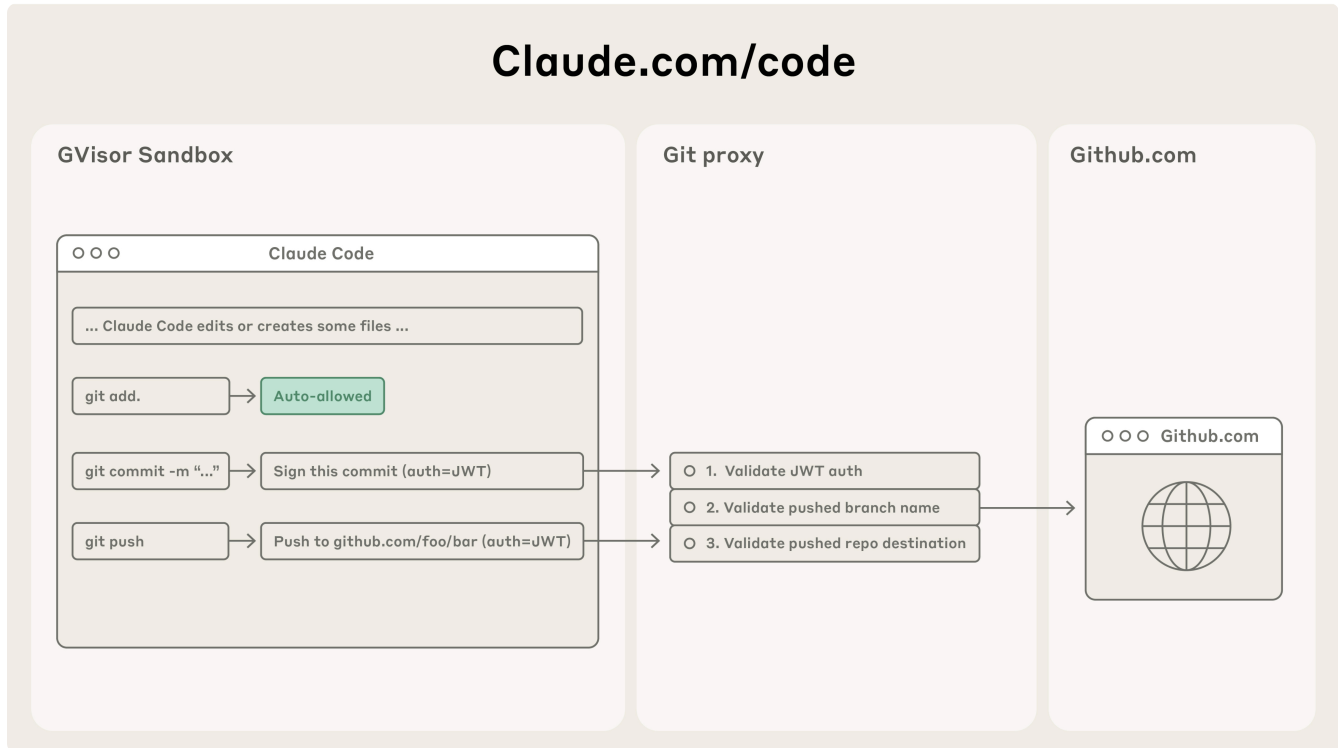
To get started with this feature, run `/sandbox` in Claude Code and check out more technical details about our security model.

To make it easier for other teams to build safer agents, we have open sourced this feature. We believe that others should consider adopting this technology for their own agents in order to enhance the security posture of their agents.

Claude Code on the web: running Claude Code securely in the cloud

Today, we're also releasing Claude Code on the web enabling users to run Claude Code in an isolated sandbox in the cloud. Claude Code on the web executes each Claude Code session in an isolated sandbox where it has full access to its server in a safe and secure way. We've designed this sandbox to ensure that sensitive credentials (such as git credentials or signing keys) are never inside the sandbox with Claude Code. This way, even if the code running in the sandbox is compromised, the user is kept safe from further harm.

Claude Code on the web uses a custom proxy service that transparently handles all git interactions. Inside the sandbox, the git client authenticates to this service with a custom-built scoped credential. The proxy verifies this credential and the contents of the git interaction (e.g. ensuring it is only pushing to the configured branch), then attaches the right authentication token before sending the request to GitHub.



Claude Code's Git integration routes commands through a secure proxy that validates authentication tokens, branch names, and repository destinations—allowing safe version control workflows while preventing unauthorized pushes.

Getting started

Our new sandboxed bash tool and Claude Code on the web offer substantial improvements in both security and productivity for developers using Claude for their engineering work.

To get started with these tools:

1. Run ``/sandbox`` in Claude and check out our docs on how to configure this sandbox.
2. Go to claude.com/code to try out Claude Code on the web.

Or, if you're building your own agents, check out our open-sourced sandboxing code, and consider integrating it into your work. We look forward to seeing what you build.

To learn more about Claude Code on the web, check out our launch blog post.

Acknowledgements

Article written by David Dworken and Oliver Weller-Davies, with contributions from Meaghan Choi, Catherine Wu, Molly Vorwerck, Alex Isken, Kier Bradwell, and Kevin Garcia